Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Ставьте “+” в чат, если да
Напишите, если есть проблемы

Вводные

Тема вебинара

Базовые элементы Kotlin

Сергей Прощаев

Преподаватель курсов «Kotlin Backend Dev.Pro», «Java Dev.Basic», «Java Dev.Pro»

Руководитель направления Java-разработки в FinTech

Telegram: @sprosh

Правила вебинара

Активно участвуем

Off-topic обсуждаем в Телеграм-канале группы

Задаем вопрос в чат или голосом

Вопросы вижу в чате, могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или задайте вопрос

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Маркетинг аналитика	Security	DevOps	Архитек-тура		
Модуль 4: Разработка API	OpenAPI	CollabAPI	Модели и маперы	Testing	Monitoring	
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin	
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде		
Модуль 9: Проектная работа	Обзор экосист.	Консуль-тация	Защита			

Маршрут вебинара

- Пакеты, модули, точки входа и др.
- Основные типы и их использование
- Null safety и Null Pointer Exception
- Основные операторы и циклы
- Исключения
- Рефлексия

Цели вебинара

После занятия вы сможете

1. Понимать базовые конструкции языка: типы, операторы, исключения и др.

Базовые элементы Kotlin

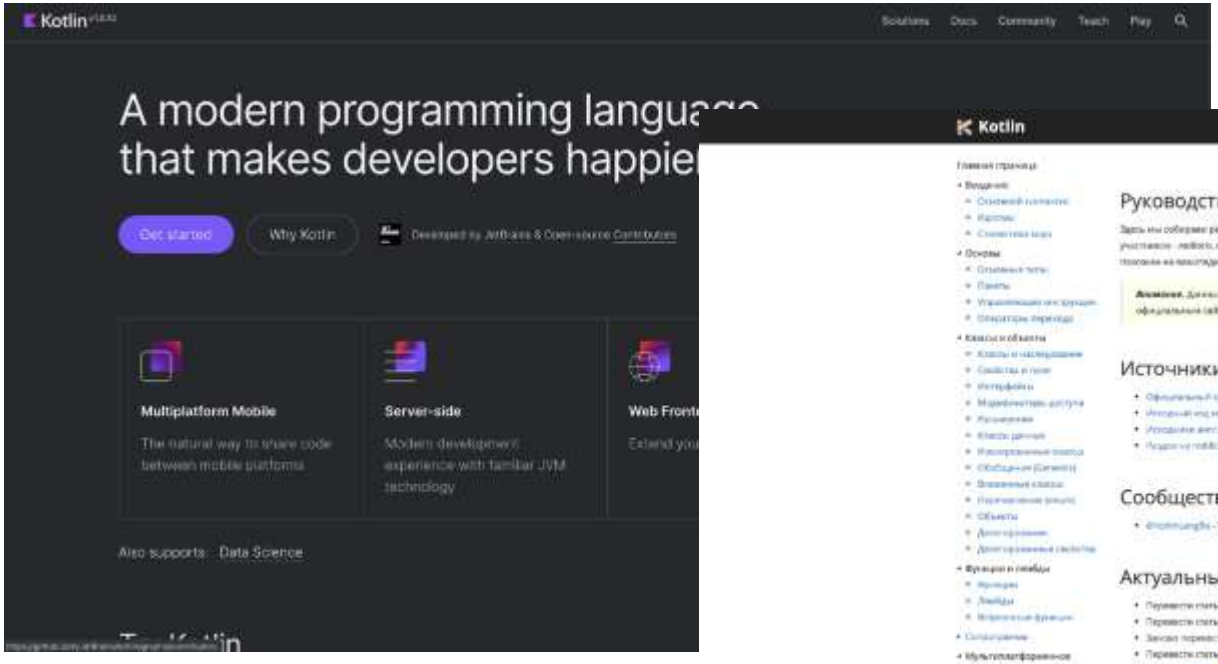
Особенности учебного модуля

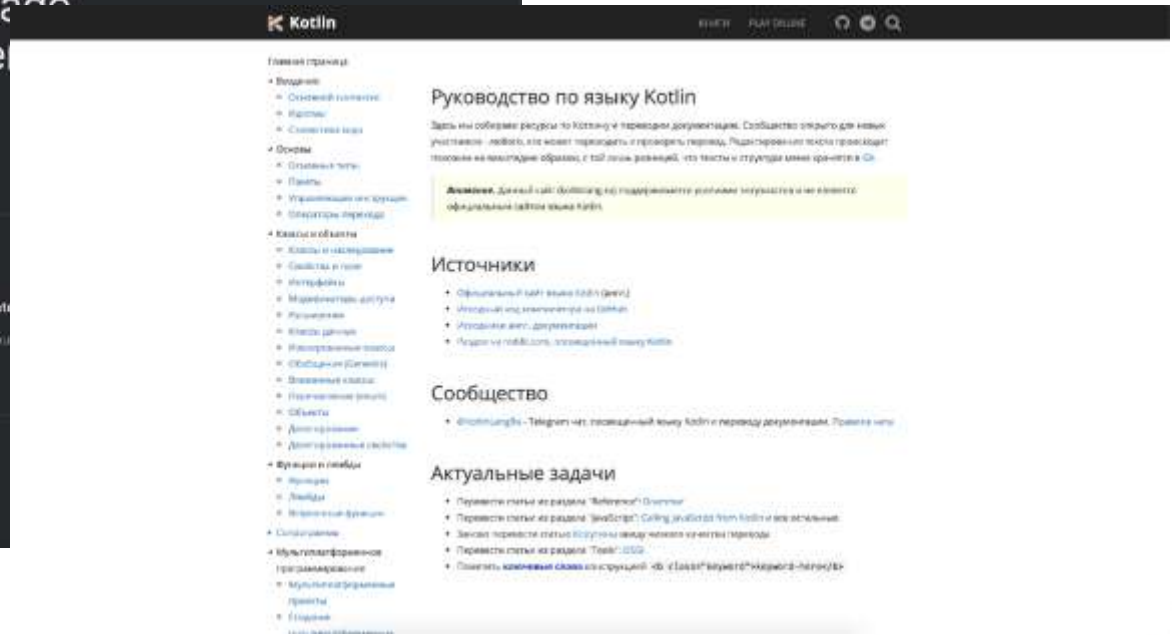
- Большой объем
- Справочный материал
- Не требуется доскональное понимание
- Будем изучать 5 месяцев
- Изучайте дополнительно

Рабочие материалы

https://kotlinlang.org

https://kotlinlang.ru





Структура программы на Kotlin

Пакеты и GroupId

Общепринятый формат – обратный домен

Пример groupId:
ru.otus.otuskotlin.marketplace
- ru.otus - Обратный домен
- Otuskotlin – группа проектов
- Marketplace - проект

Пример пакета:
ru.otus.otuskotlin.marketplace.fe.api.v1.model
- fe – группа модулей
- api.v1 – модуль
- model – подпакет

Устройство файловой системы (полное):
/src/main/kotlin/ru/otus/otuskotlin/marketplace/fe/api/v1/model

Устройство файловой системы (сокращенное):
/src/main/kotlin/model

Пакеты и GroupId

Ключевое слово **package**

Указывает пакет текущего файла

Не запрещает в одной директории иметь файлы с разными пакетами

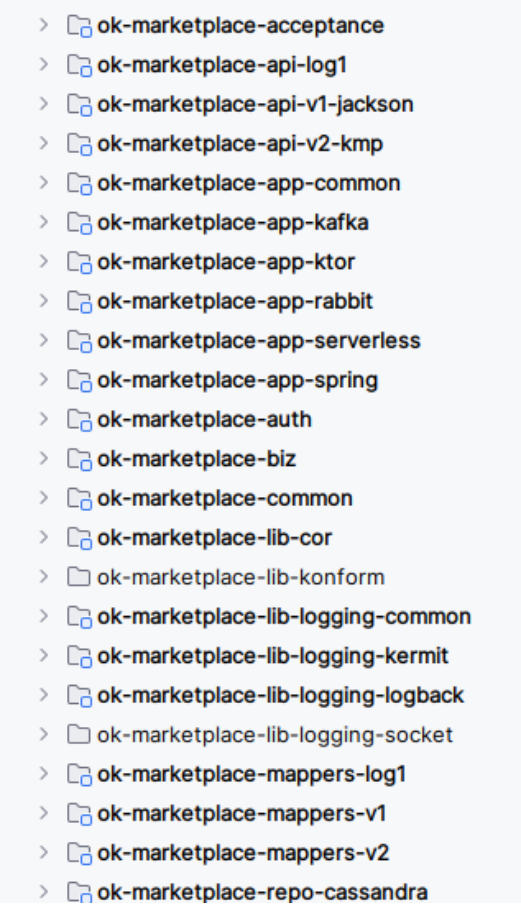
Идея кидает варнинги при отличии пакетов

```
package com.otus.otuskotlin.marketplace

fun main(args: Array<String>) {
    println("Hello World with args $args!")
}
```

Именование модулей

Особенности:

- Каждый подпроект – отдельный проект
  - Логика завязана на имя подпроекта
  - Публикация с именем пакета по умолчанию
  - Может выноситься из проекта или вноситься
  - Название должно содержать имя владельца кода
- Возможна плоская виртуальная структура подпроектов
- Подпроекты должны быть логично упорядочены



Именование классов и импорт пакетов

Любой объект программы должен быть импортирован в коде, либо содержать полный пакет

Одинаковые имена с разными пакетами импортируются с использованием as

Для избежания конфликтов, рекомендуется использовать специфическое наименование, не допускающее омонимов

Пример: MkplAdControllerV2

```
import kotlinx.coroutines.runBlocking
import kotlinx.coroutines.withTimeoutOrNull
import org.junit.AfterClass
import org.junit.BeforeClass
import org.testcontainers.containers.RabbitMQContainer
import ru.otus.otuskotlin.marketplace.api.v1.apiV1Mapper
import ru.otus.otuskotlin.marketplace.api.v1.models.AdCreateObject
import ru.otus.otuskotlin.marketplace.api.v1.models.AdCreateRequest
import ru.otus.otuskotlin.marketplace.api.v1.models.AdCreateResponse
import ru.otus.otuskotlin.marketplace.api.v1.models.AdDebug
import ru.otus.otuskotlin.marketplace.api.v1.models.AdRequestDebugMode
import ru.otus.otuskotlin.marketplace.api.v1.models.AdRequestDebugStubs
import ru.otus.otuskotlin.marketplace.api.v2.apiV2RequestSerialize
import ru.otus.otuskotlin.marketplace.api.v2.apiV2ResponseDeserialize
import ru.otus.otuskotlin.marketplace.app.rabbit.config.MkplAppSettings
import ru.otus.otuskotlin.marketplace.app.rabbit.config.RabbitApp
import ru.otus.otuskotlin.marketplace.app.rabbit.config.RabbitConfig
import ru.otus.otuskotlin.marketplace.app.rabbit.config.RabbitExchangeConfiguration
import ru.otus.otuskotlin.marketplace.stubs.MkplAdStub
import kotlin.test.*
import ru.otus.otuskotlin.marketplace.api.v2.models.AdCreateObject as AdCreateObjectV2
import ru.otus.otuskotlin.marketplace.api.v2.models.AdCreateRequest as AdCreateRequestV2
import ru.otus.otuskotlin.marketplace.api.v2.models.AdCreateResponse as AdCreateResponseV2
import ru.otus.otuskotlin.marketplace.api.v2.models.AdDebug as AdDebugV2
import ru.otus.otuskotlin.marketplace.api.v2.models.AdRequestDebugMode as AdRequestDebugModeV2
import ru.otus.otuskotlin.marketplace.api.v2.models.AdRequestDebugStubs as AdRequestDebugStubsV2
```

```
val apiV1Mapper = com.fasterxml.jackson.databind.json.JsonMapper.builder().run { th
//    configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false)
```

Точки входа

Main()

Точки входа

fun main

```
fun main() {
    println("Hello World!")
}
```

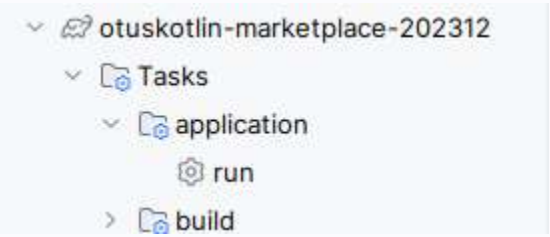
```
fun main(vararg args: String) {
    println("Hello World with args $args!")
}
```

```
fun main(args: Array<String>) {
    println("Hello World with args $args!")
}
```

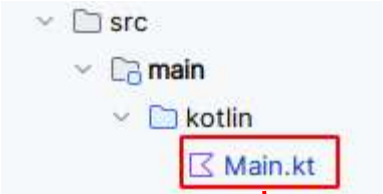
Компиляция

Требуется плагин application
Добавляет таску run

Обращение через:
<пакет>.<имя файла>**Kt**




```
fun main() {
    println("Hello World!")
}
```




```
plugins { this: PluginDependenciesSpecScope
    kotlin("jvm")
    application
}

application { this: JavaApplication
    mainClass.set("com.otus.otuskotlin.marketplace.MainKt")
}

dependencies { this: DependencyHandlerScope
```

Jar

Компиляция в jar-файл

Jar-файлы:

- Худые – только классы и ресурсы модуля
- Толстые – включает все зависимости

Толстый jar собирается с использованием доп. Плагина

Подробнее разберем на занятии по Gradle

```
sokatov@hepok:~$ java -h
Usage: java [options] <mainclass> [args...]
           (to execute a class)
   or  java [options] -jar <jarfile> [args...]
           (to execute a jar file)
   or  java [options] -m <module>[/<mainclass>] [args...]
       java [options] --module <module>[/<mainclass>] [args...]
           (to execute the main class in a module)
   or  java [options] <sourcefile> [args]
           (to execute a single source-file program)
```

```
sokatov@hepok:~$ java -jar myApplication.jar
```

Типы данных в Kotlin

Числовые типы в Kotlin

Знаковые			Беззнаковые		
Класс	Длина	Пример	Класс	Длина	Пример
Byte	8	-1.toByte()	UByte	8	1.toUByte()
Short	16	-1.toShort()	UShort	16	1.toUShort()
Int	32	-1	UInt	32	1u
Long	64	-1L	ULong	64	1uL
Float	32	-0.2f			
Double	64	-0.2			

Читабельность чисел:
1_000_000.123_123_123

Примитивных типов нет.
Оптимизация до примитивов выполняется на уровне компилятора

Преобразование типов в Котлин

Неявные преобразования типов не допускаются

```
1  val a: Int = 1
2  val b: Long = a // Type mismatch Required: Long
```

```
1  val a: Int = 1
2  val b: Long = a.toLong() // Now it's OK!
```

- toByte(): Byte
- toShort(): Short
- toInt(): Int
- toLong(): Long
- toFloat(): Float
- toDouble(): Double
- toChar(): Char

Нечисловые типы в Kotlin

Логический тип / Bool

```
val arePeopleImmortal = false
val isTheEarthRound = true
```

Символы

```
val char = 'a'
val escapeSequence = '\t' // \t, \b, \n, \r, \',
\", \\ and \$
```

Массивы / Array<T>

```
val arrayOfInt = arrayOf(1, 2, 3)
val emptyArray = emptyArray() // arrayOf()
val arrayCalculated = Array(5) { i -> (i * i) } // using index for
calculation ->
Creates an Array<String> with values ["0", "1", "4", "9", "16"]
val intArray = intArrayOf(1, 2, 3) // int[]

// same access
val element = arrayOfInt[2] // 3
val elementByFunction = arrayOfInt.get(2) // 3
```

Нечисловые типы в Kotlin

Строки. Строковые литералы

```
val string = "Hello, otus!\n"
val elementOfString: Char = string[1] // string.get(1) 'e'

val codeExample = """
    val string = "Hello, otus!\n"
    val elementOfString: Char = string[1] // string.get(1)
'e'
""".trimIndent()
```

Строковые шаблоны

```
val two = 2
val multiplicationExample = "Multiplication $two * $two = ${two * two}"
// Multiplication 2 * 2 = 4
val strExample = "String $subStr ${if (true) subStr1 else "String”}"
```

Особые типы

Any

```
val a: Any = 1
val b: Any = “Строка”
val c: Any = SomeClass()

var any: Any = 1
any = "any"
any = 3.14
```

Unit, Nothing
на следующем занятии

TypeCast: преобразование типов

```
var any: Any = 3.14

any is String // false
any is Int // false
any !is Float // true
any is Double // true

val iAmDouble = any as Double // iAmDouble: Double
val willThrow = any as String // throws ClassCastException
val willNotThrow = any as? String // null or willNotThrow: Int
```

```
var any: Any = 12

if (any is String) {
    any + "test" // smart cast to String, so String + String allowed
}
```

Smart cast to 'String' is impossible, because 'any' is a mutable property that could have been changed by this time

Cast expression 'any' to 'String' ⌥⇧⏎ More actions... ⌥⏎

Вопросы

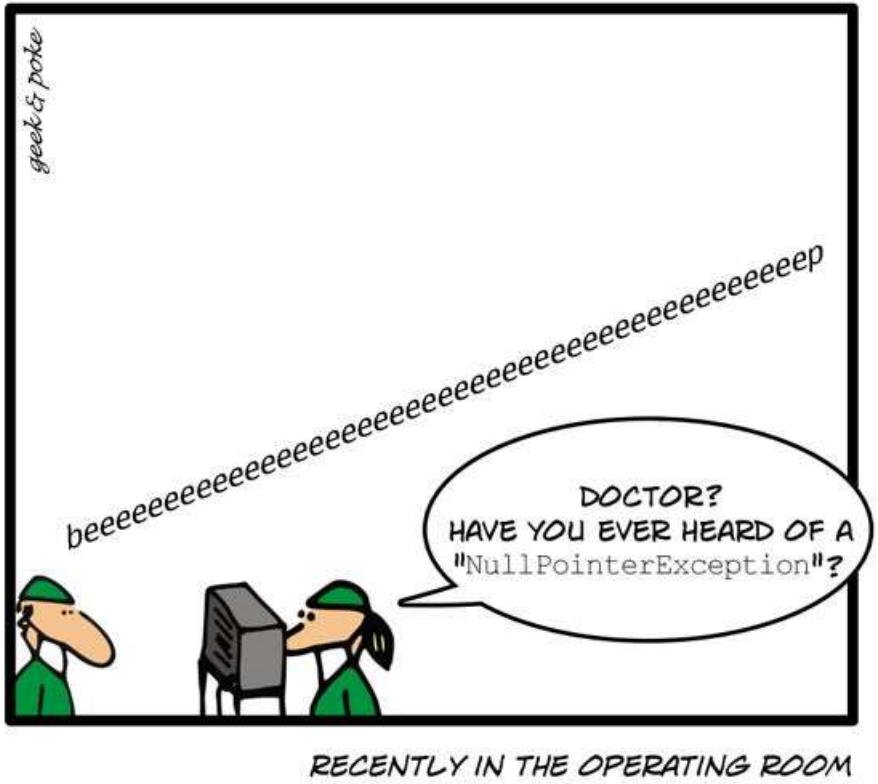
ПОВТОРИМ ИЗУЧЕННОЕ

Повторение - мать учения

```
1. val whatTypeIAm = 3 // Какой тип переменной?

2. val correctType: ?? = 3.4 // Подставить правильный тип переменной, чтобы скомпилировалось без
ошибок

3. val whatTheResult: Int = 3L // Что произойдет, если скомпилировать такой код?
(Ошибка, скомпилируется без ошибок, в переменной whatTheResult будет храниться 3, свой вариант)

4. val string = "someString"
   val charVariable: Char = string[2] // Какое значение будет храниться в переменной charVariable

5. val someAny: Any = "hello"
   someAny is Double // Какой результат такого выражения?
```

Null Safety

Null Safety



NPE в Java - 70-80% всех ошибок
NPE в Kotlin - большая редкость

Причины NPE в Котлин:

- Явное указание `throw NullPointerException()`
- Использование оператора `!!`
- Эту ошибку вызвал внешний Java-код
- Есть какое-то несоответствие при инициализации данных
- В результате рефлексии

В Kotlin строгая защита от NPE

Null Safety

Ошибка компиляции

```
1  val notNullableString: String = null // Compilation error
2  var cantBeNullable: Int = 13
3  cantBeNullable = null // haha, not too fast. Compilation error
```

Работает, как швейцарские часы

```
1  val notNullableString: String? = null
2  var canBeNullable: Int? = 13
3  canBeNullable = null // Ok. Now you won =(
```

Null Safety

1. Безопасные вызовы. "?."

```
val canBeNullable: Int? = 13
val therteenLongOrNull = canBeNullable?.toLong() // null
or 13L
```

2. Элвис оператор. "?:"

```
val longOrDefault: Long = canBeNullable?.toLong() ?: 0L
```

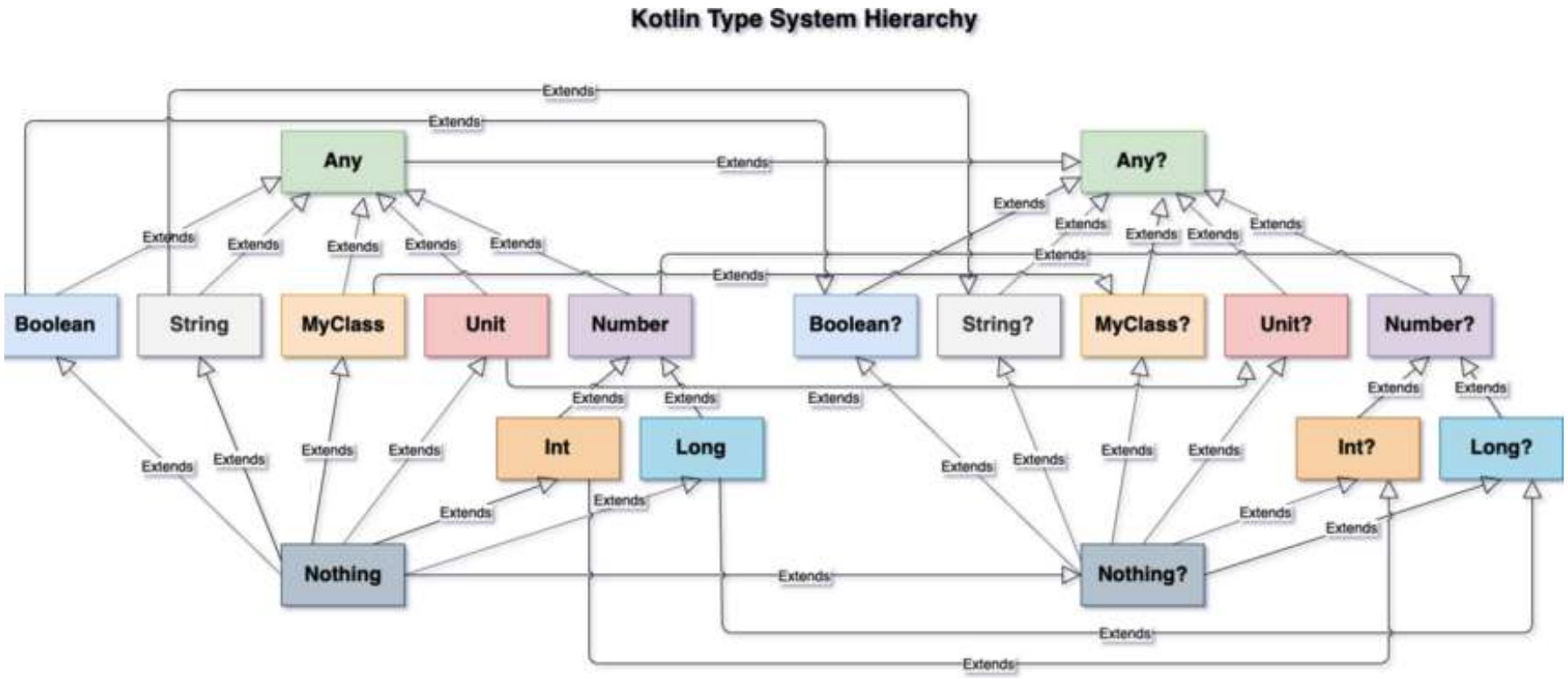
3. Оператор "!!" - Антипаттерн!

```
val nullable: String? = null

val length = nullable!!.length //NPE
```

```
val x = null

if (x != null) {
    print(x.toString())
}
```

Карта классов

Типы в Kotlin



Переменные и мутабельность

Объявление переменных

Var – мутабельная переменная
Val – иммутабельная переменная

val x = 0 // инициализация обязательна!

// Антипаттерн!
lateinit var x: String – без инициализации
If (this::x.isInitialized()) { ... }

// Ленивая инициализация (обсуждаем в след.зан.)
val x by lazy { SomeHeavyClass() }

var, val

Защита от null требует гарантий неизменности переменной

При использовании var, переменная может измениться после проверки.

Рекомендуется избегать var

```
var x: Int? = 1

fun main() {

    if (x != null) {
        x += 100
        p
    }
    x = 1

}
```

Smart cast to 'Int' is impossible, because 'x' is a mutable property that could have been changed by this time

Add non-null asserted (!!) call Alt+Shift+Enter More actions... Alt+Enter

Вопросы

ПОВТОРИМ ИЗУЧЕННОЕ

Повторение - мать учения

```
1. var nullable = null
   val whatTypeIAm = nullable?.toDouble() // Какой тип будет у переменной whatTypeIAm

2. val guessType = whatTypeIAm ?: 4.5 // Какой тип и значение будет у переменной guessType

3. var nullable = null
   val guessType2 = nullable?.toLong() ?: 5 // Какой тип и значение будет у переменной guessType2

4. val guessType3 = guessType?.toInt() ?: 2 // Какой тип и значение будет у переменной guessType3

5. var anyType: Any = 12
   val castFromAny = anyType as? String // Какой тип и значение будет у переменной castFromAny
```

Операторы в Kotlin

Поддерживаемые операторы в Kotlin

```
val long = 1L + 3 // long: Long = 4


// +, -, *, /, % for any number types
// + operator for basic types
//(Byte, Short, Int, Long, Float, Double)
operator fun plus(other: Byte): Int
operator fun plus(other: Short): Int
operator fun plus(other: Int): Int
operator fun plus(other: Long): Long
operator fun plus(other: Float): Float
operator fun plus(other: Double): Double

// for string concatenation
operator fun String?.plus(other: Any?): String

// division
val first = 1
val second = 2
first / second // first.div(second)

// +=
var a = 1
a += 1 // a.plusAssign(1)

// plus for string
var string = "test"
string += "test" // string.plus("test")
```

https://kotlinlang.org/docs/keyword-reference.html#operators-and-special-symbols

Числовые

математические: +, -, *, /, =, ++, --, +=, …

Логические: &&, ||, !, == (по equals), === (по ссылке), <, >, <=, >=

битовые: shl, shr, and, or, xor, inv, ...

строковые: + (конкантенация)

Управление потоком исполнения. Условные операторы и циклы

Условные операторы.

Условный оператор IF

```
var sum = 100
var currentValue = 10
val summator = 20

// Simple
if (currentValue < sum) currentValue += summator

// With else
if (currentValue < sum) {
    currentValue += summator
} else {
    currentValue -= summator
}

// As expression
val newValue = if (currentValue < sum) currentValue + summator else currentValue - summator

// As expression with code block
val newValue = if (currentValue < sum) {
    currentValue + summator
} else {
    currentValue - summator
}
```

Условные операторы

Условное выражение When

```
1  when {
2      x.isOdd() -> print("x is odd")
3      x.isEven() -> print("x is even")
4      else -> print("x is funny")
5  }
```

```
1  val x: Int = 10
2  val s: String = "10"
3  when (x) {
4      Integer.parseInt(s) -> print("s=\"$s\" encodes to x=$x") // s="10" encodes to x=10
5      else -> print("$s doesn't encode to $x")
6  }
```

```
1  val number = 11
2  val test = when (val multiply = number * 10) {
3      in 10..100 -> "$multiply <= 10"
4      in 100..1000 -> "$multiply > 10"
5      else -> "Multiply is too much for me"
6  }
```

Условные операторы и циклы

Цикл For (используется редко)

```
1  val helloOtus = "hello, otus!"
2  for(ch in helloOtus) {
3      print(ch)
4  } // will print "hello, otus!"
```

```
1  val array = arrayOf(1, 2, 3)
2
3  for ((index, element) in array.withIndex()) {
4      print("element at index $index is $element")
5  }
```

Условные операторы и циклы

Цикл While

```
1  var counter = 10
2  while(counter > 0) { // var counter = 10 > 0 not working
3      counter--
4  }
```

Цикл do while

```
1  var counter = 0
2  do {
3      counter += 1
4      val y = counter * 2
5  } while (y > 3) // y available from that place
```

Исключения

Exceptions (Исключения)

Исключения – способ обработки ошибок

В Kotlin нет checked exceptions

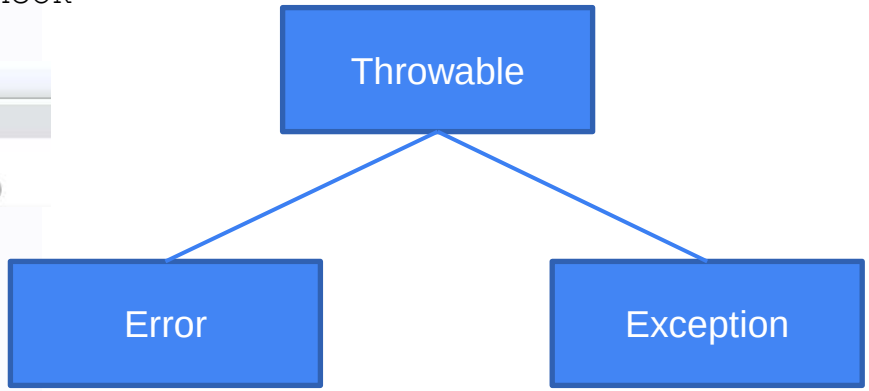
Список наиболее известных исключений и ошибок

```
throw kotlin.Exception
```

- Exception (kotlin)
- RuntimeException (kotlin)
- ArithmeticException (kotlin)
- ClassCastException (kotlin)
- ConcurrentModificationException (kotlin)
- IllegalArgumentException (kotlin)
- IllegalStateException (kotlin)
- IndexOutOfBoundsException (kotlin)
- KotlinNullPointerException (kotlin)
- NoSuchElementException (kotlin)
- NoWhenBranchMatchedException (kotlin)

```
throw kotlin.Err
```

- Error (kotlin)
- AssertionError (kotlin)
- NotImplementedError (kotlin)



Генерация и перехват исключений

throw – поднять исключение

catch – поймать исключение
соответствующего типа

finally – выполняется при любом исходе

```
try {
    throw RuntimeException("test")
} catch (e: RuntimeException) {
    println("Caught $e")
} catch (e: Throwable) {
    println("Caught $e")
} finally {
    println("All done")
}
```

Генерация и перехват исключений

throw – поднять исключение

catch – поймать исключение
соответствующего типа

finally – выполняется при любом исходе

```
try {
    throw RuntimeException("test")
} catch (e: RuntimeException) {
    println("Caught $e")
} catch (e: Throwable) {
    println("Caught $e")
} finally {
    println("All done")
}
```

Как выражение

```
val res :String = try {
    "str"
} catch (e: RuntimeException) {
    "exc1"
} catch (e: Throwable) {
    "exc2"
} finally {
    println("All done")
}
```

Вопросы

Рефлексия

Цели вебинара

Проверка достижения целей

1. Понимать базовые конструкции языка: типы, операторы, исключения и др.

Список материалов для изучения

- https://kotlinlang.org/
- https://gradle.org/
- https://github.com/otuskotlin/202411-ok-marketplace

Рефлексия

С какими основными мыслями
и инсайтами уходите с вебинара?

Как будете применять на практике то,
что узнали на вебинаре?

Следующий вебинар

Модуль 1: Введение в Kotlin

Функциональные элементы Kotlin

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание! Приходите на следующие вебинары